

SISTEMAS PROGRAMABLES

delay() vs millis(): en Arduino

Reporte de práctica

Integrantes del equipo:

Daniel Bustamante

Iván Luan

Materia: Sistemas Programables

Plataforma: Arduino UNO R4 WiFi

1. Introducción

En el desarrollo de sistemas embebidos con Arduino, uno de los retos más comunes es controlar el tiempo en que ocurren los eventos sin detener la ejecución del resto del programa. Las dos formas más habituales de manejar temporización son la función `delay()` y la función `millis()`. Este reporte compara ambos enfoques a partir de una práctica de laboratorio en la que se controló un conjunto de tres LEDs, simulando un semáforo vehicular.

2. Objetivo

Comprender las diferencias entre el uso de `delay()` y `millis()` para el control de temporización en Arduino, identificando ventajas, limitaciones y escenarios de aplicación de cada una, mediante la implementación práctica de un circuito de tres LEDs.

3. Marco teórico

3.1 La función `delay()`

`delay(ms)` es una función que detiene por completo la ejecución del programa durante la cantidad de milisegundos indicada. Mientras el microcontrolador está "dormido" en esa pausa, no puede leer entradas, actualizar otras salidas ni ejecutar ninguna otra tarea. Es sencilla de usar, pero bloqueante.

3.2 La función `millis()`

`millis()` devuelve la cantidad de milisegundos transcurridos desde que el Arduino se encendió. En lugar de pausar el programa, se guarda una marca de tiempo de referencia y en cada iteración del `loop()` se compara el tiempo actual contra esa referencia; cuando la diferencia alcanza el intervalo deseado, se ejecuta la acción correspondiente. De esta manera el programa nunca se detiene y puede atender múltiples tareas de forma simultánea (no bloqueante).

4. Desarrollo práctico

Se implementó el control de tres LEDs (rojo, amarillo y verde) conectados a las terminales digitales D2, D3 y D4 respectivamente, simulando la secuencia de un semáforo vehicular: verde 8 segundos, amarillo 2 segundos y rojo 6 segundos.

4.1 Conexiones utilizadas

- Rojo → pin D2, con resistencia de 220–330 Ω , cátodo a GND
- Amarillo → pin D3, con resistencia de 220–330 Ω , cátodo a GND
- Verde → pin D4, con resistencia de 220–330 Ω , cátodo a GND

4.2 Versión con delay()

El siguiente código enciende cada LED en secuencia utilizando delay() para controlar la duración de cada color:

```
// Semáforo de 3 LEDs
// Arduino UNO R4 WiFi
// Usa delay()

const int ROJO = 2;
const int AMARILLO = 3;
const int VERDE = 4;

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AMARILLO, OUTPUT);
  pinMode(VERDE, OUTPUT);
}

void loop() {
  digitalWrite(VERDE, HIGH);
  digitalWrite(AMARILLO, LOW);
  digitalWrite(ROJO, LOW);
  delay(8000);

  digitalWrite(VERDE, LOW);
  digitalWrite(AMARILLO, HIGH);
  digitalWrite(ROJO, LOW);
  delay(2000);

  digitalWrite(VERDE, LOW);
  digitalWrite(AMARILLO, LOW);
  digitalWrite(ROJO, HIGH);
  delay(6000);
}
```

4.3 Versión con millis()

El siguiente código realiza exactamente la misma secuencia, pero sin bloquear el programa: se utiliza una variable de estado (estado) y una marca de tiempo (tiempoAnterior) para decidir cuándo avanzar al siguiente color.

```
// Semáforo de 3 LEDs usando millis()
// Arduino UNO R4 WiFi

const int ROJO = 2;
const int AMARILLO = 3;
const int VERDE = 4;

const unsigned long TIEMPO_VERDE = 8000;
```

```

const unsigned long TIEMPO_AMARILLO = 2000;
const unsigned long TIEMPO_ROJO = 6000;

unsigned long tiempoAnterior = 0;
int estado = 0; // 0=Verde 1=Amarillo 2=Rojo

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AMARILLO, OUTPUT);
  pinMode(VERDE, OUTPUT);
  digitalWrite(VERDE, HIGH);
  tiempoAnterior = millis();
}

void loop() {
  unsigned long tiempoActual = millis();

  if (estado == 0 && tiempoActual - tiempoAnterior >= TIEMPO_VERDE) {
    estado = 1;
    tiempoAnterior = tiempoActual;
    digitalWrite(VERDE, LOW);
    digitalWrite(AMARILLO, HIGH);
  }
  else if (estado == 1 && tiempoActual - tiempoAnterior >= TIEMPO_AMARILLO) {
    estado = 2;
    tiempoAnterior = tiempoActual;
    digitalWrite(AMARILLO, LOW);
    digitalWrite(ROJO, HIGH);
  }
  else if (estado == 2 && tiempoActual - tiempoAnterior >= TIEMPO_ROJO) {
    estado = 0;
    tiempoAnterior = tiempoActual;
    digitalWrite(ROJO, LOW);
    digitalWrite(VERDE, HIGH);
  }
}

```

5. Análisis comparativo

A partir de las dos implementaciones, se identificaron las siguientes diferencias clave:

Aspecto	delay()	millis()
Bloqueo del programa	Sí, detiene por completo la ejecución durante el tiempo indicado.	No, el programa sigue ejecutándose; solo se compara el tiempo transcurrido.
Lectura de sensores/botones	No es posible durante la pausa.	Se puede leer en cada vuelta del loop(), sin perder eventos.
Múltiples tareas simultáneas	Muy difícil o imposible de forma independiente.	Permite manejar varias temporizaciones a la vez de forma independiente.

Precisión a largo plazo	Aceptable para pruebas simples.	Igual de precisa; además no se desincroniza por dependencias entre delays.
Complejidad del código	Muy simple de escribir y leer.	Requiere variables de estado y comparación de tiempos.
Uso recomendado	Pruebas rápidas, prototipos sin necesidad de reaccionar a eventos.	Proyectos con botones, sensores o múltiples procesos (máquinas de estado).

6. Conclusiones

Ambas funciones cumplen el objetivo de temporizar eventos, pero tienen usos distintos. `delay()` es adecuada para pruebas rápidas o circuitos muy simples donde no se necesita que el Arduino reaccione a otros eventos mientras espera. `millis()`, en cambio, es la opción recomendada para proyectos reales de sistemas programables, ya que permite mantener el programa "despierto" en todo momento, leer botones o sensores sin perder pulsaciones y controlar varios procesos de manera independiente, como se requiere en máquinas de estados finitos más complejas (por ejemplo, un semáforo con solicitud peatonal).

Como equipo concluimos que dominar `millis()` es indispensable para avanzar hacia sistemas embebidos más robustos, ya que evita los problemas de bloqueo que `delay()` introduce en aplicaciones interactivas.